

结论

当前状态：服务器端已跑通 Linux 仿真，耗时约 53 小时。配置为 CPU: C906, SoC: smart_run, 系统: Linux, 仿真器: Verilator。小范围实验中已验证升级仿真器版本可带来约 3 倍单线程提速。

问题：多线程加速不可行，单线程优化可行。53h 的启动时间通过升级 Verilator 版本可缩短到约 18h。

多线程不可行的原因：来自两个方面：C906 芯片设计本身 + Verilator 仿真器调度机制，两者叠加导致多线程不仅不加速，反而更慢。香山团队的实验性仿真器 GSIM 声称能达到更高的单线程速度，但适配 C906 有一定工作量，待验证。

问题1：当前模型的特点：电路依赖紧密

C906 是单核顺序处理器，电路内部有大量前后依赖。Verilator 5.020 在处理当前 C906/smart_run 模型时抑制了 117 条 UNOPTFLAT 警告，升级版本到 5.050 后降到了 6 条。说明仿真器发现了一些难以充分优化的组合逻辑结构。这类结构会让生成的仿真程序做更多重复计算，因此可能拖慢仿真。

在默认的 2 线程构建中，Verilator 5.020 生成了 21 个 mtask，5.050 生成了 25 个。mtask 是仿真器把整个芯片电路拆分出的可调度任务——拆得越多说明能独立计算的部分越多。作为参照，香山处理器的电路能拆出 8436-12275 个 mtask，C906 只有 25 个，少了三个数量级，说明能拆开的独立工作非常少。

更关键的是，这 25 个 mtask 之间还有 54 条依赖边，平均每个 mtask 和 2-3 个其他 mtask 有前后依赖关系。有依赖就意味着必须等对方算完才能开始，不能同时执行。仿真器的静态成本模型估算出理论并行度只有 3 倍——即使用满所有依赖关系允许的并行，最多也只能 3 倍。

仿真器编译 C906 电路后的统计数据

指标	Verilator 5.020	Verilator 5.050
当前构建生成的 mtask 数量	21	25
mtask 之间的依赖连线	40	54
静态成本模型估算的并行度	4 倍	3 倍
抑制的 UNOPTFLAT 警告	117 条	6 条

实测数据：线程越多越慢

Verilator 5.020（当前生产版本）

线程数	稳态速度 (retired/s)	相对单线程
1	913	1.00x
2	552	0.60x
4	375	0.41x

Verilator 5.050（升级仿真器版本）

线程数	稳态速度 (retired/s)	相对 5.020 单线程
1	2708	2.97x
2	1274	1.40x
4	988	1.08x

优化尝试

实验	稳态速度 (retired/s)	结论
5.050 2线程 + 负载引导优化	1593	比无优化快 25%，但仍只有单线程的 59%
5.050 2线程，限制最多8个 mtask	2031	实际生成 6 个 mtask 时最好，仍比单线程慢 25%
5.050 2线程，限制最多2个 mtask	2622	实际只生成 1 个 mtask，退化成单线程
5.050 单线程 + <code>-march=native</code>	2302	反而慢 15%

调度任务拆得越细越慢，越粗越快，证明同步开销远超并行收益。

问题2：Verilator 的问题：多线程同步开销

Verilator 多线程变慢并非 C906 独有，社区中也有类似报告：

- [verilator#2913](#) - 一个小设计使用 `--threads 4` 后，运行时间约为 `--no-threads` 的 4 倍。
- [verilator#2590](#) - 一个原本看起来容易并行的设计使用 `--threads 2` 后，运行时间反而增加到约 2 倍。
- [Verilator 官方文档](#) - mtask 增多会增加运行时通信和同步开销；调度器会因此尽量避免生成过多 mtask。

多线程仿真的每个仿真周期中，工作线程需要领取任务；遇到前后依赖时，需要等其他线程完成；最后还要汇合，才能进入下一步。只有每份任务足够大、可以连续计算较长时间时，分工节省的时间才会超过交接和等待的成本。这是 Verilator 多线程调度的固有机制，社区 issue 中的其他设计也遇到了同样的问题。

两个问题叠加的效果

C906 的电路只能拆出 25 个 mtask，且彼此依赖密集（54 条边）。Verilator 的多线程调度需要频繁领取任务、等待依赖、汇合同步。两者叠加的结果是：线程刚开始工作，很快就要停下来等待或汇合。第二个核心虽然参与了计算，却没有缩短总时间，反而增加了交接成本。

负载引导优化虽然减少了一部分等待，仍然没有快过单线程；把任务合并到最后只剩一个时，速度又接近单线程。这说明当前瓶颈是没有足够大、足够独立的工作可以交给多个核心持续执行。

当前基线指标

服务器长跑 (Verilator 5.020 单线程, 已验证成功)

- CPU: Intel Xeon Gold 6248R @ 3.00GHz, 64核
- 启动时间: 2026-09-01 12:00
- 到达 PID 1: 2026-09-03 17:00 (约 53 小时)
- 总退休指令数量: 约 100M

本机实验 (i9-14900HX/WSL2, 2026-09-03)

- Verilator 5.020 单线程: 913 retired/s
- Verilator 5.050 单线程: 2708 retired/s (快 2.97 倍)
- 预计 5.050 跑到 PID 1: 约 18 小时 (53h / 2.97)

单线程提速方向

多线程问题较大, 但单线程效率仍有提升空间:

- **Verilator 版本升级:** 5.020 → 5.050 单线程快 3 倍 (913 → 2708 retired/s), 纯编译器优化, 已验证。
- **GSIM:** 香山团队的 [GSIM](#) 通过跳过当前周期不需要重新计算的电路来减少工作量, 不依赖增加仿真线程。 [DAC 2025 论文](#) 报告, 在香山启动 Linux 时比 Verilator 快 7.34 倍, 在 Rocket 运行 CoreMark 时快 19.94 倍。这是两个特定设计和负载的结果, 不能直接当作 C906 的预期加速倍数。GSIM 当前接受 CHIRRTL 输入, 现成支持 ysyx3、Rocket、BOOM 和 XiangShan; 它不能直接读取 C906 的 Verilog。适配 C906 需要增加或寻找可靠的输入转换路径, 并重新验证测试平台和 Linux 启动结果, 工作量和最终收益都不确定。